Gila Monster
Pulse generator v1.0

To meet the need for a pulse generator capable of triggering external instruments in neurostimulation protocols with multiple frequency patterns, the Gila Monster v1.0 was developed. Although numerous DIY solutions are available in online repositories, this project intentionally adopts a minimalist hardware design, centered on an Arduino Uno built around the ATmega328P microcontroller. The design prioritizes the timing engine at the hardware and firmware levels, ensuring that time-critical pulse generation is isolated from user interface operations. Encoder polling and LCD updates are handled via interrupts and non-blocking routines so that UI-related tasks do not introduce latency or timing jitter into the output signal

1. Firmware Specifications

- Stimulation Modes: Multi-engine pulse generation supporting Continuous (periodic), Burst (trains with inter-burst gaps), and Non-Periodic Stimulation (NPS) with randomized intra-window intervals.
- Time Base & Granularity: Microsecond-level timing relying on a 16 MHz system clock. Due to AVR hardware prescaling, the fundamental temporal resolution (software polling step) advances in 4 μs increments.
- Frequency Range & Resolution: Programmable output frequency from 0.1 Hz to 500 Hz, with a 0.01 Hz resolution capability for precise slow-wave protocols.
- Pulse Width (PW) Range: Configurable pulse durations. The recommended reliable lower bound is 50 μs in NPS mode, where pulse timing is governed by a blocking delayMicroseconds() call. In Continuous and Burst modes, the firmware enforces a minimum of 100 μs, subject to ISR (Interrupt Service Routine) preemption overhead. Under encoder-idle conditions the worst-case actual pulse width is bounded at ~108 μs against the programmed 100 μs, representing an 8% deviation across the full frequency range. The theoretical worst-case of ~350 μs could only reachable during encoder interaction while in ON state. **For a full statistical analysis of pulse width accuracy, see Section 6.**
- Output Control & Switching Latency: Direct port manipulation using PORTD, bit 7 (PD7) to minimize software overhead, achieving a theoretical minimum switching latency of ≈125 ns (limited by the ATmega328P single instruction cycle).
- Output Logic Level: 5 V TTL-compatible digital output.
- External Triggering & Synchronization: Integrated hardware interrupt support via Pin D3 (INT1), allowing synchronization with external lab equipment. The system supports user-selectable Rising or Falling edge detection with a sub-microsecond hardware response latency. Note that the interrupt service routine includes a hardware interlock that rejects incoming trigger events while the output pin is actively HIGH. Consequently, trigger pulses arriving during an ongoing stimulation pulse will be silently discarded. This behavior is by design to prevent pulse overlap, but may result in missed synchronization events at high stimulation frequencies or narrow inter-trigger intervals.
- Single-Shot Diagnostic: Dedicated hardware-level manual pulse capability via Pin D6. Governed by a software-based debounce routine and a logical safety interlock that physically prevents manual firing while the primary stimulation engine is active (State: ON).
- Duty Cycle Safety Clamp: Automated logic constraint that prevents 100% duty cycle (DC latching). If the requested PW exceeds the period, the firmware enforces a mandatory low-phase cutoff (PWmax = T−10 μs).
- Parameter Storage: Non-volatile EEPROM integration used to store 13 user-configurable operational parameters, preserving experimental setups across power cycles.
- Serial Communication: UART-based serial interface (9600 baud) for logging and reporting of operational state and elapsed timestamps, enabling external data acquisition, real-time monitoring, and protocol reproducibility.

2. Operating Engines

2.1 Deterministic Engine (Continuous [Periodic] / Burst Modes)

The deterministic timing engine computes the output period (**T**) as a direct function of the programmed frequency (**f**), according to:

$$\mathrm{T}\mu\mathrm{s} = \frac{1{,}000{,}000}{f\mathrm{Hz}}$$

In **Continuous Mode**, pulses are generated at a fixed period with cycle-accurate timing.

In **Burst Mode**, pulse delivery is governed by:

- a **pulse counter** defining the number of pulses per train, and a secondary **silent-state timer (Gap)** that enforces the inter-train interval (ITI).

2.2 Non-Periodic Stimulation (NPS) Engine

The Non-Periodic Stimulation (NPS) engine implements a constrained uniform distribution algorithm to generate temporally irregular pulse trains while preserving overall pulse density. Although the underlying random selection is uniform within constrained temporal bounds, the sequential dependency between events produces a heavy-tailed distribution of inter-stimulus intervals (ISI). As a result, shorter ISIs occur more frequently while longer intervals remain possible, yielding a power-law-like temporal structure rather than a purely Poisson or periodic pattern.

- **Algorithm:**

$$t_{\mathrm{n}} = \mathrm{Random}\ (t_{\mathrm{n}-1} + \mathrm{Min}, \mathrm{Remaining\ Window\ Space})$$

- **Window Size:** Fixed **1,000 ms** temporal window.
- **Pulse Density:** Constant **N pulses per window** (Pulse Density Modulation).
- **Temporal Structure:** Variable inter-stimulus intervals (ISI) exhibiting a **power-law-like distribution**, increasing temporal unpredictability.

The NPS engine operates independently of the deterministic timing core, ensuring that the event scheduling does not compromise microsecond-level pulse timing accuracy

References by Cota, et al.

https://doi.org/10.1016/j.yebeh.2019.106609
https://doi.org/10.1016/j.yebeh.2008.09.006

3. Parameter Matrix & Data Mapping

ID	Parameter	Type	Logic / Range	Resolution
00	Mode	Option	Manual (0) / Triggered (1)	N/A
01	Trigger Edge	Option	Falling (0) / Rising (1)	N/A
02	Output Type	Option	Continuous (0) / Burst (1) / NPS (2)	N/A
03	State	Action	OFF / ON	N/A
04	Frequency	Decimal	0.10 Hz – 500.00 Hz *(Cont. and Burst mode only)*	0.01 Hz
05	Pulse Count	Integer	1 – 999 pulses *(Burst mode)* / 1 – 50 pulses *(NPS mode)*	1 pulse
06	Gap (ITI)	Integer	0 – 999,999 ms *(Burst mode only)*	1 ms
07	Minimum ITI	Decimal	0.00 – 999.99 ms *(NPS mode only)*	0.01 ms
08	Pulse Width Mode	Option	Fixed (0) / Duty Cicle (1)	N/A
09	Pulse Width	Decimal	Fixed (*µ*s) or Duty Cycle (%)	0.01 ms / %
10	Timer	Integer	0 (Disabled) – 999,999 ms	1 ms
11	Save	Action	“Saved”	N/A
12	Comm	Action	“Sent”	N/A

4. Hardware Interface (Pinout)

Digital & Analog Pin Assignment

MCU Pin	Arduino Pin	Direction	Pull-Up	Function	Notes
PD7	D7	Output	Disabled	Pulse Output	High-speed TTL output. i.e. use 220 Ω series resistor for opto-isolation or long cables.
PD3	D3	Input	Disabled	Hardware Trigger (INT1)	External interrupt input. **Sub-*µ*s latency**
PD2	D2	Input	Enabled	Encoder Channel A (CLK)	Quadrature decoding. Internal pull-up reduces noise and wiring complexity.
PD5	D5	Input	Enabled	Encoder Channel B (DT)	Quadrature decoding. Matched with Channel A.
PD4	D4	Input	Enabled	Encoder Push Button (SW)	Active-low user input. Debounced in firmware.
PD6	D6	Input	Enable	Single Shot Button	Manual pulse fire. Debounced (200ms) and interlocked with master state
PC4	A4	I/O	Enabled	I²C SDA	Shared data line. Standard-mode **100 kHz**. External pull-ups optional if LCD provides them.
PC5	A5	Output	Enabled	I²C SCL	Shared clock line. Standard-mode **100 kHz**.

---

5. Serial Communication & Data Validation

The **COMM [12]** function operates as a **diagnostic and validation interface**, providing access to internal configuration and timing behavior via the serial port.

1. **Protocol:** UART serial communication at **9600 baud, 8-N-1**.
2. **Current state-machine configuration and active timing parameters**.
3. **Monte Carlo Validation (NPS Mode):** When operating in **Non-Periodic Stimulation (NPS)** mode, the firmware executes an internal **Monte Carlo–style simulation** over the programmed timer duration. The resulting **inter-trial interval (ITI) sequence** enable offline statistical analysis and verification of ISI variability.

Notes for open-hardware users

- Simulation and data export are decoupled from the real-time timing engine and do not affect pulse generation.

---

6. Operational Constraints and Architectural Limitations

1. Firmware Logic & Safety Constraints

- **Pulse Width Clamping:** If the programmed pulse width (PW) exceeds the pulse period (T), the firmware automatically clamps PW to T−10 μs. This ensures a mandatory low phase between pulses, preventing a constant-high (latched) output state and protecting the biological preparation.
- **Mode-Dependent Minimum Pulse Width:** The firmware enforces a mode-specific minimum pulse width floor. In NPS mode, pulse timing is governed by a blocking delayMicroseconds() call, making 50 μs a reliable and precise lower bound. In Continuous and Burst modes, pulse width is enforced via polling of the micros() function, subject to ISR preemption overhead. The firmware enforces a minimum of 100 μs in these modes. Under encoder-idle conditions — the recommended operational state — the idle ISR execution overhead is approximately 8 μs, bounding the worst-case actual pulse width to 108 μs against the programmed 100 μs. The theoretical worst-case of 350 μs (100 μs + full ISR overhead of 250 μs) is only reachable during encoder interaction while in ON state.
- **NPS Timing Prioritization:** In Non-Periodic Stimulation (NPS) mode, the cumulative minimum timing constraint must satisfy $N \times ITImin < 1000$ ms, where N is the requested number of pulses per window. If this condition is violated, the engine enforces a fail-safe prioritization of the minimum inter-stimulus interval, automatically reducing the effective pulse count within the window to preserve temporal validity.
- **NPS Pulse Count Hard Limit:** In NPS mode, the maximum number of pulses per window is hardware-constrained to 50. This limit is enforced by a statically allocated array (nps_timestamps[50]) residing in the ATmega328P's 2 KB SRAM. Increasing this value would risk memory exhaustion and undefined behavior.
- **Single-Shot Diagnostic Logic:** The Single-Shot button (D6) is only operational when the device is in a non-generating state (State: OFF). Upon detection of a valid HIGH to LOW transition, the firmware bypasses the menu-based timing core to deliver an immediate pulse. The duration of this manual pulse is fixed at 10 ms for diagnostic and calibration purposes.

2. Timing Architecture & Hardware-Induced Limitations

- **Polling-Based Timing Dependency:** Instead of utilizing deterministic Hardware Timers (such as CTC mode), the main algorithm relies on a continuous polling loop of the micros() function. Consequently, the effective temporal resolution is dictated by the execution time of the longest instruction block in loop(), rather than the 16 MHz microcontroller clock. Any blocking in the main routine induces jitter in the output pin's state switching.
- **Critical Concurrency and ISR Overhead:** The system uses hardware interrupts on Timer1 to ensure debouncing and the capture of asynchronous encoder events, configured via Timer1.initialize(250). This forces the processor to preempt the main loop every 250 μs. If a pulse's temporal switching aligns exactly with this preemption window, the stimulus pin's deactivation will remain pending until the ISR returns. This overhead is the root cause of the mode-dependent minimum pulse width constraints described above.

---

3. Interface Latency & Biological Safety Risks

- **Synchronous I2C Bus Latency:** The primary temporal bottleneck is the user interface, specifically the LiquidCrystal_I2C library. The I2C bus imposes significant synchronous data transmission latency. If the main loop schedules a signal transition (falling edge) during an I2C transmission, the CPU will fail to process the state change until the transaction is complete, resulting in severe artificial pulse stretching.
- **Dynamic Interface Bypass (High-Fidelity Regime):** To mitigate I2C latency without altering the hardware topology, the firmware implements a state control flag (val_change) coupled to the rotary encoder. When the system is idle without human interaction, this flag remains false, fully bypassing the LCD screen update. In this regime, execution speed is maximized, allowing for high-fidelity timing for frequencies between 0.1 Hz and 500 Hz with stabilized pulse widths.
- **Safety Risk (The "Stop-Click" Hazard):** The most severe architectural risk manifests during the manual interruption of the stimulus. Pressing the button to toggle the state to OFF immediately transitions the control flag to true, triggering the blocking LCD screen updates. If the stimulation pin is actively injecting current (HIGH logic state) at this exact microsecond, the CPU will be held up in the display's I2C update before executing the cutoff command. This causes an erratic prolongation of tens of milliseconds in the last pulse.

---

4. Recommended Operating Guidelines

High-Fidelity Operation: To guarantee maximum temporal accuracy and prevent erratic pulse stretching caused by I2C update interruptions, the rotary encoder **must not be** manipulated while active stimulation (ON Mode) is running. Any parameterization adjustments must be strictly performed in standby (OFF Mode).

Pulse Width Selection: For Continuous and Burst modes, users should treat the configured pulse width as a nominal value rather than a guaranteed duration. Under encoder-idle conditions the worst-case pulse extension is bounded by the idle ISR execution overhead of ~8 μs, resulting in a maximum actual pulse width of 108 μs against the programmed 100 μs. This 8% deviation is statistically predictable across the full frequency range of the device. The worst-case absolute scenario of 350 μs (nominal 100 μs + full ISR overhead of 250 μs) is only theoretically reachable during encoder interaction while the system is in ON state. Protocol design should incorporate the nominal 8 μs tolerance, particularly when charge delivery is a safety-critical parameter.

5. Pulse Width Temporal Accuracy:

The following analysis quantifies the expected pulse width deviation under nominal conditions, defined as active stimulation. Under this condition, the ISR (Interrupt Service Routine) executes every 250 μs but performs only a minimal pin-state comparison, with an estimated execution overhead of approximately 8 μs. The probability of ISR preemption occurring during a pulse falling-edge transition is governed by the ratio between the polling resolution (4 μs) and the ISR period (250 μs), **yielding a per-cycle risk of 1.6%.** When preemption occurs, the worst-case pulse extension is bounded by the idle ISR execution time of ~8 μs, resulting in an actual pulse width of 108 μs against the programmed 100 μs, a deviation of 8%.

System Parameters (Encoder-Idle):

Parameter	Value
ISR period	250 μs
ISR execution overhead (idle)	~8 μs
Programmed pulse width	100 μs
Polling resolution	4 μs
Vulnerability window	4 μs
Risk per cycle	1.6%
Worst-case pulse width	108 μs
Worst-case deviation	8%

Frequency-Dependent Impact:

Parameter	1 Hz	10 Hz	100 Hz	500 Hz
Period	1,000,000 μs	100,000 μs	10,000 μs	2,000 μs
Cycles/min	60	600	6,000	30,000
Affected cycles/min	~1	~10	~96	~480
Worst-case pulse	108 μs	108 μs	108 μs	108 μs
Pulse deviation	8%	8%	8%	8%
Duty cycle nominal	0.01%	0.10%	1.00%	5.00%
Duty cycle worst-case	0.011%	0.11%	1.08%	5.40%

In both cases, the absolute pulse deviation remains constant at 8 μs, and the duty cycle impact remains below 0.1 percentage points, statistically predictable for standard protocols. The primary risk factor remains user interaction with the rotary encoder during active stimulation, which can introduce substantially higher ISR overhead and should be strictly avoided during ON state operation, as described in the **Recommended Operating Guidelines**.

6. Stop-Click Hazard: Frequency and Pulse Width Dependent Impact

The Stop-Click risk is governed by two independent factors: the probability of pressing the stop button during an active HIGH pulse, and the resulting pulse extension caused by I2C blocking. The probability is determined solely by the duty cycle, while the extension is a fixed hardware characteristic of the I2C bus transaction.

System Parameters (Stop-Click):

Parameter	Value
I2C transaction overhead	~2 ms
I2C clock	100 kHz
Pulse extension (worst case)	+2 ms

Frequency-Dependent Impact (e.g.: Fixed Pulse Width = 5 ms):

Parameter	1 Hz	10 Hz	20 Hz	100 Hz
Period	1,000,000 μs	100,000 μs	50,000 μs	10,000 μs
Pulse width	5 ms	5 ms	5 ms	5 ms
Duty cycle	0.5%	5%	10%	50%
Stop-Click probability	0.5%	5%	10%	50%
Nominal pulse	5 ms	5 ms	5 ms	5 ms
Worst-case pulse	~7 ms	~7 ms	~7 ms	~7 ms
Pulse deviation	+40%	+40%	+40%	+40%

7. Operation Modes

7.1 Continuous Mode (Periodic)

Generates a constant flow of pulses at a fixed frequency. This is the simplest operational mode, suitable for protocols requiring regular, periodic stimulation without temporal structure.

Example Configuration: 20 Hz stimulation with 5 ms pulse width.

Parameter	Value
Type	Cont.
Freq	020.00 Hz
Width	Fixed Time
Pulse	005.00 ms
State	ON

ISR Pulse Width Accuracy (encoder-idle):

Programmed pulse	Worst-case pulse	Deviation
5 ms	5.008 ms	~0.16%

Stop-Click Hazard:

Duty cycle	Stop-Click probability	Nominal pulse	Worst-case pulse	Deviation
10%	10%	5 ms	~7 ms	+40%

7.2 Burst Mode (Sequenced Trains)

Generates groups of pulses (trains) separated by a configurable silence interval (Gap). Suitable for protocols requiring episodic stimulation with controlled inter-train intervals.

Example Configuration: 10 pulses at 50 Hz with 2 ms pulse width, repeating every 2 seconds.

Parameter	Value
Type	Burst
Freq	050.00 Hz
Count	010
Gap	001500 ms
Pulse	002.00 ms
State	ON

Note: Gap + Burst duration = Cycle time. At 50 Hz, 10 pulses span 200 ms. With Gap = 1500 ms, total cycle = 1700 ms.

ISR Pulse Width Accuracy (encoder-idle):

Programmed pulse	Worst-case pulse	Deviation
2 ms	2.008 ms	~0.4%

Stop-Click Hazard:

Duty cycle	Stop-Click probability	Nominal pulse	Worst-case pulse	Deviation
10%	10%	2 ms	~4 ms	+100%

Recommended: Stop-Click risk is highest at the beginning of a train. Preferred interruption point is during the Gap (silent interval), where the output pin is LOW and I2C blocking carries no pulse extension risk.

7.3 NPS Mode (Non-Periodic Stimulation)

Generates a specific number of pulses per 1,000 ms window at randomized intervals. Suitable for protocols requiring temporal unpredictability while preserving overall pulse density.

Example Configuration: 4 random pulses per second with 100 μs pulse width, at least 20 ms between pulses.

Parameter	Value
Type	NPS
Count	004
ITImin	020.00 ms
Pulse	000.10 ms
State	ON

NPS Constraint Validation:

$N \times ITImin = 4 \times 20\ ms = 80\ ms < 1000\ ms$

Pulse count within hard limit: $4 \leq 50$

ISR Pulse Width Accuracy:

Mode	Programmed pulse	Worst-case pulse	Deviation
NPS (blocking)	0.10 ms	0.10 ms	0%

In NPS mode, pulse timing is governed by delayMicroseconds(). ISR preemption does not affect pulse width accuracy.

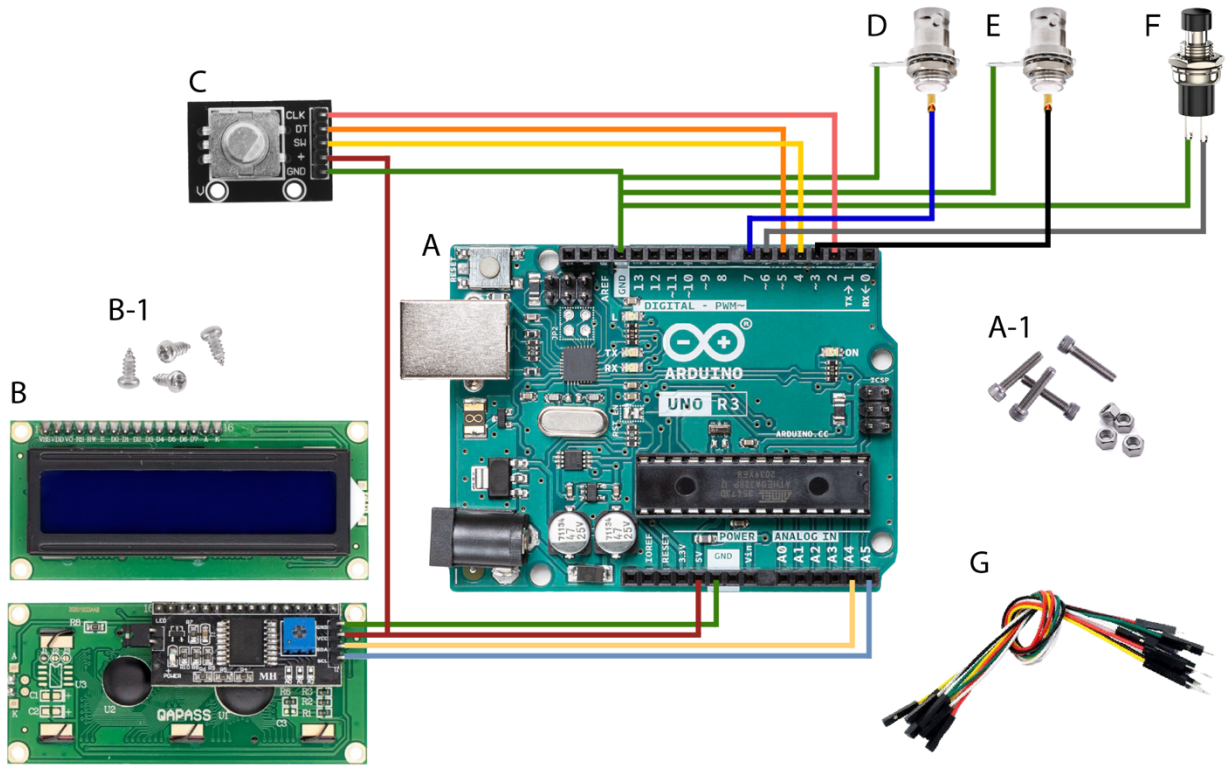
Stop-Click Hazard:

Duty cycle	Stop-Click probability	Nominal pulse	Worst-case pulse	Deviation
~0.04%	~0.04%	0.10 ms	~2.1 ms	+2000%

Although the Stop-Click probability is very low in NPS mode due to the short pulse width, the relative deviation is extreme. The absolute extension of ~2 ms is fixed regardless of the programmed pulse width. Use of the Timer parameter is strongly recommended.

8. Assembly Instructions

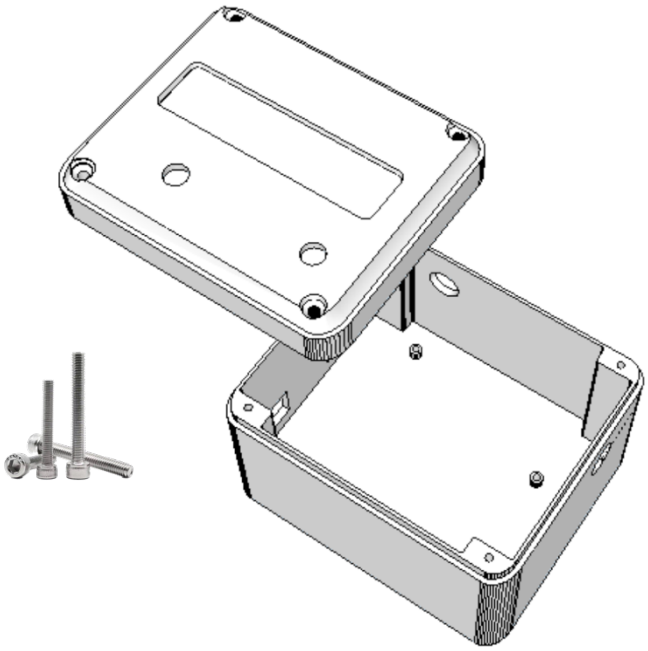
8.1 Electronics



	Materials	Quantity	Link	$
A	Arduino Uno R3	1	Mouser electronics	25.87
A-1	M2 x 12 mm Allen Bolt Socket Cap Screws M2 Hex Nut for Arduino	4	-	-
B	LCD module 16x2, (WxHxD): 80x36x13 mm I2C LCD display module	1	Amazon	10
B-1	Round Head Tap Screw for LCD Ø 2-2.5 mm, L = 3-6 mm)	4	-	-
C	360 Rotatory encoder module KY-040	1	Amazon	~7 (2 un)
D, E	Female BNC panel-mount	2	-	-
F	Push Button RS PBS 110	1	Amazon	~.60
G	Cables, different colors (e.g. Ø 2.5 mm, cross section 0,5 mm², length 10-15 cm)	~15	Amazon	~5 (40 un)

8.2 3D parts

- PLA case (WxHxD): 7.5 x 6 x 8.5 cm
- 4 M2 x 20 mm Allen Bolt Socket Cap Screws



PSSDS '04